

Every capability in the platform, in one place.

The platform briefing makes the case for Netcare. This document backs it up — a complete, module-by-module list of what's actually built and working today, for anyone on your side who wants the full picture before signing off.

12	29	74	6
CAPABILITY GROUPS	BACKEND MODULES	DISTINCT FEATURES	ROLES, UNLIMITED CUSTOM

Identity, access & sessions	8	Roles & permissions	6
Platform console	3	Hospital onboarding & config	9
Front desk & scheduling	7	Clinical care	6
Pharmacy & inventory	8	Diagnostics (lab)	4
Maternity & child health	5	Billing & payments	8
Trust, consent & notifications	6	Platform conventions	4

01 · IDENTITY, ACCESS & SESSIONS

Signing in, and staying signed in safely

One login screen for every role, on every hospital's own subdomain.

Identifier-based login

public

Sign in with an email or a phone number — no role tab, no hospital picker. The identifier alone resolves who's asking and which hospital they belong to.

Self-service registration public (patient) · admin-created (staff)	A patient can register directly; staff accounts (doctor, nurse, lab, pharmacist) are created by their hospital's admin, not self-signed-up.
Session refresh self-scoped	A session extends itself without forcing a password re-entry.
Sign out — one device or all self-scoped	Log out the current session, or every session on the account at once, from a lost or shared device.
Login throttling applied pre-auth, by identifier + IP	Repeated failed attempts against one identifier or IP address are slowed automatically, before it becomes a brute-force problem.
Change password self-scoped	From inside the account, with the current password required first.
Forgot / reset password public	An email-based reset flow with a time-limited, single-use token.
Live "who am I" self-scoped	The signed-in user's role, permissions and hospital are re-checked on demand — a revoked grant is reflected without a fresh login.

02 · ROLES & PERMISSIONS

Access control your superadmin runs, not engineering

The authorization engine underneath every screen in the product.

Full permission catalog roles.manage	Every capability in the system is listed and browsable — nothing is a hidden, undocumented check.
Custom roles roles.manage	A superadmin builds a role from scratch — not a pick from a fixed list of five.
Per-permission scope roles.manage	Every grant carries its own scope, "own" or "all" — a nurse seeing only her own recorded vitals versus a lab tech seeing the whole hospital's orders are the same permission, different scope.

Assignable-roles list

`users.manage`

The "create a user" screen only offers roles that make sense to hand out from that account — not the entire catalog, including ones above its own level.

Edit or retire a role

`roles.manage`

Change what a role can do, or remove it outright, without touching code.

Audit log

`audit.read`

A running, queryable record of who did what and when, across the hospital.

03 • PLATFORM CONSOLE

What the platform owner sees, that no hospital does

A separate, superadmin-only layer that sits above every tenant.

Cross-hospital overview

`platform.read`

Headline numbers rolled up across every hospital on the platform, in one screen.

Cross-hospital drill-down

`platform.read`

Inspect any hospital's patients, doctors, appointments, departments or user list without switching subdomains.

Platform-level correction

`patients.manage`

Fix a record across tenant lines on the rare occasion a hospital genuinely needs it — through its own permission, not a backdoor around normal tenant isolation.

04 • HOSPITAL ONBOARDING & CONFIGURATION

Standing a new hospital up, and keeping its paperwork straight

Everything that happens before, and after, a hospital goes live.

Guided onboarding wizard

`hospitals.manage`

Collects legal identity, category, statutory licences and the first admin account in one flow.

One rule for what to ask

`hospitals.manage`

A single endpoint tells the wizard which licences and documents apply to this hospital's category — the frontend never re-derives or duplicates that logic.

Per-hospital module toggles

`hospitals.manage`

Lab, pharmacy, nursing, payments, medical records, telemedicine and ANC are each switched on or off per hospital — inherited from its category template at onboarding, adjustable after. A module that's off doesn't just hide in the menu: its permissions, consent purposes and licence prompts disappear with it.

Licence expiry tracking

`hospitals.manage`

A standing report of licences approaching expiry, so a renewal never gets missed silently.

Public hospital directory

`public`

A lightweight listing endpoint for a marketing or discovery surface.

Hospital settings, self-service

`hospital.profile.read / .manage`

An admin edits their own hospital's profile, contacts and operational config directly — no ticket to the platform team.

Branding assets

`hospital.profile.manage`

Upload or remove a letterhead (checked for A4 proportions and print resolution) and a logo.

Print header, composed once

`tenant-scoped`

One function builds the header for an invoice, a lab report or a prescription — letterhead artwork if uploaded, otherwise the hospital's name, address and phone as text so nothing ever prints anonymous. The tax id (GSTIN) is included only on the invoice, which needs it to be valid — a lab report never receives it, because a document doesn't get to see data it has no use for.

Full hospital lifecycle

`hospitals.manage`

A superadmin can list, onboard, inspect, update or delete any hospital — deletes cascade cleanly through everything that hospital ever created.

Booking, departments and the calendar

What runs the waiting room and the diary.

Departments

`departments.read` / `.manage`

Category-aware department list, created and edited by the hospital's own admin.

Appointments

`appointments.read` / `.create` /
`.manage` / `.delete`

Book, list, inspect, update or cancel — with a live stats endpoint feeding dashboard counts directly.

One booking per patient, per department, per day

enforced inside
`appointments.create`

Enforced on the server, not just a warning in the UI — the second attempt is refused outright.

Doctor availability, without the patient list

`appointments.create`

A dedicated endpoint returns which slots are open and which are taken — never who's booked into them.

Schedule blocks

`schedule.read` / `.manage`

A doctor blocks out their own unavailable time directly.

Consultation fees

`fees.read` / `.manage`

Set per doctor or per department, feeding billing automatically at checkout.

Video consults

`video_consults.join` ·
`schedule.manage`

Scheduled slots that can be created, booked and removed independent of in-person scheduling.

06 · CLINICAL CARE

The patient record, and everything written to it

One chart per patient, read the same way by every role that's allowed to.

Patient records

`patients.read` / `.manage` / `.delete`

Look a patient up by id or by their linked user account, update their details, and — platform-gated — delete one.

Patient-scoped sub-views

5 separate `.read` permissions

A patient's own appointments, medical records, payments, prescriptions and vitals — each checked against its own permission, not just "is this your patient."

Doctor records

`doctors.read` / `.delete`

Look a doctor up by id or linked account, see their own appointment list, update their profile, and — platform-gated — delete one.

Medical records

`medical_records.read` / `.manage`

Create and update a longitudinal clinical record per patient.

Prescriptions

`prescriptions.read` / `.manage` / `.delete`

Create, edit or retire a prescription — always attributed to the doctor who is actually signed in, never a typed-in name.

Vitals

`vitals.read` / `.record` / `.delete`

Recorded by nurses, editable, and — permission-gated — deletable.

07 • PHARMACY & INVENTORY

Stock that moves with real orders, not a guess

Oral medicines and injectables, each with their own ledger.

Medicine catalog

`medicines.read` / `.manage` / `.delete`

Create, edit or retire the drugs a hospital stocks.

Inventory movement ledger

`inventory.read` / `.manage`

Every restock and adjustment, queryable — a full paper trail of what moved and when.

Low-stock report

`inventory.read`

Medicines below their reorder threshold, surfaced automatically rather than discovered at the counter.

Medication orders, as four distinct steps

`medication_orders.read` / `.manage` / `.dispense` / `.administer`

Create the order, dispense it, administer it, or cancel it — four separately-permissioned actions, not one big "manage."

Expiry- and stock-aware dispensing

`medication_orders.dispense`

Dispensing is refused outright if there isn't enough stock, or if the stock on hand has expired.

Bill straight from dispensing

`medication_orders.dispense`

A dispensed order can be billed directly from the pharmacy screen.

Injectables, on their own ledger

`injectables.read / .manage / .delete`

A separate catalog, low-stock report, and restock/adjust history — never mixed with oral medicine stock.

Injection orders

`injection_orders.read / .manage / .administer / .delete`

Create, administer, cancel or delete — mirroring the medication-order workflow for injectable drugs specifically.

08 • DIAGNOSTICS (LAB)

Ordering a test, and standing behind the result

A separate review step between "run" and "released."

Lab test catalog

`lab_tests.read / .manage / .delete`

The tests a hospital offers, managed independently of any single order.

Test orders

`lab_orders.read / .create / .process / .delete`

Create, inspect, update, cancel or delete an order end to end.

A review step before a result ships

`lab_orders.review`

Reviewing a result is its own permission, separate from processing one — a second check between a test being run and a patient seeing it.

Test results

`lab_reports.read · lab_orders.process`

Enter or amend a result against its order.

09 • MATERNITY & CHILD HEALTH

Pregnancy through infancy, on one linked record

Purpose-built for maternity hospitals, and available to any multi-specialty one.

Pregnancy records

`pregnancies.read` / `.manage`

Create, inspect and update — with an ANC-visit summary attached automatically wherever the record is viewed.

ANC visits

`pregnancies.read` / `.manage`

Logged against a pregnancy over time, building the visit history the summary draws from.

Babies

`babies.read` / `.manage`

Linked to the mother's record from birth — not a separate, disconnected patient file.

Growth measurements

`babies.read` / `.manage`

Logged against a baby over time, tracked against standard growth curves.

Immunizations

`babies.read` / `.manage`

Scheduled ahead of time, then marked as given once administered.

10 • BILLING & PAYMENTS

Money in, invoiced by the server

Three billing surfaces, one pricing rule and one payment engine underneath.

Payments

`payments.read` / `.create` / `.manage`

Create, list and update a payment record.

Pricing engine — one answer to "what does this cost"

used by `appointments.create`

A consultation's price lives on the hospital's own fee schedule, keyed by visit type — New Patient, Follow-up, Emergency, or whatever a hospital adds — never on the doctor, and never in the request body. A visit type still priced at zero blocks booking rather than silently billing nothing.

Consultation billing summary

`payments.read`

A rollup purpose-built for the consultation side of a hospital's revenue.

Pharmacy billing summary

`payments.read`

A rollup purpose-built for dispensed-medicine charges.

Combined injectable & lab billing

`payments.read`

One summary spanning both, so a single visit doesn't produce two disconnected invoices.

Invoices

`payments.read`

Generated and retrieved from the server — never assembled or totaled on the client.

Per-hospital payment credentials

`hospital.profile.manage`

A hospital can run its own Razorpay account — its own key and secret, stored on its profile — or fall back to the platform's shared account if it hasn't set one up yet. Either way, nobody but that hospital's admin can see or change which one is in use.

Razorpay integration

`appointments.create`

Initiate a payment and verify its signature server-side before it's ever treated as paid.

11 • TRUST, CONSENT & NOTIFICATIONS

The parts that protect the patient, not just the hospital

Consent, and the record of what happened.

Built around Indian data-privacy law, not a generic checkbox. The consent model exists because of four specific requirements: [DPDP Act 2023](#) (consent must be free, specific, informed, itemised and as easy to withdraw as to give), [DPDP s.9](#) (verifiable parental or guardian consent under age 18 — not an edge case in a product built around newborn and paediatric records), the [Telemedicine Practice Guidelines 2020](#) (a teleconsultation the patient initiated carries implied consent; one the practitioner initiated needs an explicit one, and the two are recorded differently), and the [IT/SPDI Rules 2011](#) (written consent for health data specifically). None of this is asserted as a legal compliance certification — it's what the code in `app/consent.py` is actually built to do, available for your own legal team to verify.

Consent purposes catalog

`public within tenant`

The specific reasons consent can be asked for, defined once and reused everywhere it's captured — itemised on purpose, so "treatment" and "marketing" are separately refusable, never bundled into one tickbox. A purpose tied to a module a hospital doesn't run (telemedicine, for one that never does video consults) simply never appears — showing a notice for processing that won't happen would be misleading, not thorough.

Consents, granted and withdrawn

`consents.read / .manage`

Withdrawing consent writes a withdrawal timestamp rather than deleting the row — "they consented and later withdrew" and "they never consented" are different facts, and only one of them is a defence. Tied to the person, not to a patient record, so a staff member's own data-protection rights have somewhere to live too.

Minor & guardian consent

`consents.manage`

Age is checked against the day consent is given. Under 18 routes to guardian consent automatically, per DPDP s.9 — not a policy someone has to remember to apply.

Teleconsult consent, by method

`consents.manage`

A video consultation the patient books carries implied consent; one the practitioner initiates requires an explicit one first — the two are stored as distinct methods, matching the Telemedicine Practice Guidelines exactly rather than treating all video visits the same.

Push notifications, tied to real events

`self-scoped (token)`

Firebase-based alerts fire from the actual event, not a generic reminder job: an appointment booked or rescheduled notifies both the patient and the doctor; a lab order notifies the assigned role when it's placed and the patient plus doctor when the result is ready; a medication or injection order notifies the dispensing role when it's created and the patient and doctor once it's dispensed or administered.

Audit trail

`audit.read`

Who did what, and when — retained independently of the record it describes, even after that record is gone. Not foreign-keyed to what it logs, for the same reason a consent withdrawal isn't a deletion: the proof has to outlive the thing it's proof of.

The guarantees that hold everywhere, not just in one screen

Cross-cutting rules applied uniformly, so no single endpoint can quietly be the exception.

One list convention, everywhere

Every collection endpoint takes the same three query parameters — free-text search, page size, offset — declared once so they can't drift in name or behavior between screens. Learn the pattern on appointments and it works identically on patients, medicines, or any other list in the product.

A ceiling on page size

A client asking for an enormous page can't quietly turn pagination off — the server caps how many rows come back in one request, regardless of what's asked for.

Tenant integrity on writes, not just reads

Tenant scoping stops a hospital from reading another's rows; a separate check stops it from writing a reference to one. A create or update whose body names another tenant's department, doctor or patient id is refused outright — 404, the same answer as if the id never existed, so the check itself gives nothing away.

A domain resolves to at most one tenant, deliberately

The platform's own apex domain is excluded from ever resolving as a hospital's subdomain by name collision. A superadmin, who has no home tenant, always targets one explicitly — never by a guess that happens to match.
